Temporal IC Knowledge Graph — Demo Script

"Déjà Vu of Design: 30 Years of Open-Source Processor Evolution"

Database: `ic-knowledge-graph-temporal`
ArangoDB UI: https://qmlz4af1.rnd.pilot.arango.ai
Estimated runtime: 20–30 minutes
Audience: Technical stakeholders, hardware engineers, EDA researchers

Setup — Before You Present

> ⚠ **Caution**
>
> **You MUST switch databases before running any query.**
> In the ArangoDB Web UI, the database selector is at the **very top of the left sidebar**, just below the ArangoDB logo. It defaults to `ic-knowledge-graph`. Click it and select **`ic-knowledge-graph-temporal`**. All temporal collections (`DesignEpoch`, `DesignSituation`, `RTL_Module`, etc.) live there — not in the main DB.

1. Open the ArangoDB Web UI and **switch to database `ic-knowledge-graph-temporal`** (top-left sidebar dropdown)
2. Open **Queries** (left nav → Queries icon)
3. Keep this script open in a second window; paste each query when ready

What's in the DB:

Item	Count
Repositories ingested	4 (OR1200, mor1kx, marocchino, ibex)

Item	Count
Total commits	3,796
RTL modules (across all history)	6,404
Named design epochs	381
Design situations	721
Authors	137
Author→Module MAINTAINS edges	615
Semantic bridges (RESOLVED_TO)	214
Cross-repo structural bridges	72
Cross-repo evolutionary bridges	1
Golden Entities (OR1200 / MOR1KX / IBEX / MAROCCHINO)	170 / 39 / 546 / 120

Scene 1 — "The Living Graph" (~3 min)

Talking point: *Traditional graphs capture a snapshot. This graph captures 30 years of evolution. Every RTL module exists in time — it has a birthday and sometimes a death date.*

Query 1a — Show the scope

```
// How much history do we have?
LET repos = (
  FOR c IN GitCommit
    COLLECT r = c.repo WITH COUNT INTO n
    RETURN {repo: r, commits: n}
)
LET epochs = (FOR e IN DesignEpoch COLLECT r = e.repo WITH COUNT INTO n RETURN {repo: r
RETURN {
  total_commits:    SUM(repos[*].commits),
  total_modules:    LENGTH(RTL_Module),
  total_epochs:     LENGTH(DesignEpoch),
  total_situations: LENGTH(DesignSituation),
  total_authors:    LENGTH(Author),
  semantic_bridges: LENGTH(RESOLVED_TO),
  cross_repo_links: LENGTH(CROSS_REPO_SIMILAR_TO),
  by_repo: repos
}
```

Expected output: 3,796 commits, 6,404 modules, 381 epochs, 721 situations, 137 authors, 214 semantic bridges, 72 cross-repo links.

Fix — Backfill git_tag on existing DesignEpoch nodes

Run this once in the UI to fix the null git_tags. The ETL code has been fixed so future ingest runs will populate git_tag correctly.

```
// Derive git_tag from the milestone_ epoch label (e.g. milestone_5_2 → "5.2")
FOR e IN DesignEpoch
  FILTER e.epoch_type == "milestone_tag"
  LET tag_raw = REGEX_REPLACE(e.label, "^.+milestone_", "")
  LET tag = REGEX_REPLACE(tag_raw, "_", ".", true)
  UPDATE e WITH { git_tag: tag } IN DesignEpoch
  RETURN { epoch: NEW.label, git_tag: NEW.git_tag }
```

Query 1b — Epoch timeline as a graph

Important

Two separate query contexts in ArangoDB — use the right one:

- **Standalone Query Editor** (left nav → Queries icon → New Query): Supports synthetic `{vertices, edges}` objects. Switch to the **Graph** tab in results to visualise.
- **Graph Explorer built-in query panel** (Graphs → `IC_Temporal_Knowledge_Graph` → Queries): Stricter — only accepts `_id` strings pointing to real graph collections. Use this to find seed nodes, then click-expand.

For the Standalone Query Editor — paste this, run it, then click the **Graph** tab in the result:

```
// Render the mor1kx epoch skeleton as a timeline arc
// (skeleton only: milestone + period + initial — skips 90+ major_refactors)
LET repo_node = {
  _id:   "synthetic/mor1kx_repo",
  _key:  "mor1kx_repo",
  label: "openrisc/mor1kx",
  type:  "Repo"
}

LET epoch_vertices = (
  FOR e IN DesignEpoch
    FILTER e.repo == "openrisc/mor1kx.git"
    FILTER e.epoch_type == "milestone_tag"
        OR e.epoch_type == "initial_commit"
        OR STARTS_WITH(SPLIT(e.label, " — ")[1], "period_")
    SORT e.start_ts ASC
    RETURN MERGE(e, {
      label: e.git_tag != null ? e.git_tag : SPLIT(e.label, " — ")[1]
    })
)

LET edges = (
  FOR i IN 0..LENGTH(epoch_vertices)-1
    LET e = epoch_vertices[i]
    LET from_id = i == 0 ? repo_node._id : epoch_vertices[i-1]._id
    RETURN {
      _id:   CONCAT("synthetic/edge_", i),
      _key:  CONCAT("edge_", i),
      _from: from_id,
      _to:   e._id,
      label: DATE_FORMAT(e.start_ts * 1000, "%yyyy-%mm")
    }
)

RETURN {
  vertices: APPEND([repo_node], epoch_vertices),
  edges: edges
}
```

For the Graph Explorer query panel — use this instead (returns seed `_id` values, then click-expand):

```
FOR e IN DesignEpoch
  FILTER e.repo == "openrisc/mor1kx.git"
  FILTER e.epoch_type == "milestone_tag"
      OR e.epoch_type == "initial_commit"
      OR STARTS_WITH(SPLIT(e.label, " — ")[1], "period_")
  SORT e.start_ts ASC
  RETURN e._id
```

Point out: Milestone tags like `v5.2` are automatically detected from git. Time-period epochs (`period_2013_09`) fill the gaps between releases.

Scene 2 — "The Time Machine" (~4 min)

Talking point: *OR1200 has a great backstory — it was originally in SVN. The git history starts in May 2009 when the community migrated to github. We can ask: "What did OR1200 look like the moment it arrived on GitHub?" And then: "How many modules were added in the big rel3 drop in June 2010?"*

Query 2a — State of OR1200 at GitHub migration day (May 2009)

```
// What RTL modules existed in OR1200 on its first GitHub commit (May 2009)?
LET target_ts = 1243238472   // 2009-05-25 — first git commit (SVN migration)
FOR m IN RTL_Module
  FILTER m.repo == "openrisc/or1200.git"
  FILTER m.valid_from_ts <= target_ts
  FILTER m.valid_to_ts > target_ts
  SORT m.label ASC
  RETURN {
    module: m.label,
    epoch:  m.design_epoch,
    file:   m.file
  }
```

Expected: ~60 modules — the full OR1200 v1 design that arrived on GitHub.

Query 2a-alt — State after the OR1200 rel3 major release (June 2010)

```
// What changed when the rel3 snapshot landed?
LET before_ts = 1243238472   // 2009-05-25 — initial import
LET after_ts  = 1283211145   // 2010-08-30 — rel3 fully merged

LET before = (FOR m IN RTL_Module FILTER m.repo == "openrisc/or1200.git"
  FILTER m.valid_from_ts <= before_ts AND m.valid_to_ts > before_ts
  RETURN m.label)

LET after = (FOR m IN RTL_Module FILTER m.repo == "openrisc/or1200.git"
  FILTER m.valid_from_ts <= after_ts AND m.valid_to_ts > after_ts
  RETURN m.label)

RETURN {
  modules_at_svn_migration: LENGTH(before),
  modules_after_rel3:        LENGTH(after),
  new_in_rel3:               MINUS(after, before),
  removed_in_rel3:           MINUS(before, after)
}
```

Point out: `MINUS()` shows which modules were added and removed between the two snapshots — a built-in, zero-effort diff between two points in time.

Query 2b — Show ibex module growth over time

```
// How did ibex grow? Module count at each epoch boundary
FOR e IN DesignEpoch
  FILTER e.repo == "lowRISC/ibex.git"
  LET module_count = LENGTH(
    FOR m IN RTL_Module
      FILTER m.repo == "lowRISC/ibex.git"
      FILTER m.valid_from_ts <= (e.end_ts != null ? e.end_ts : 9999999999)
      FILTER m.valid_to_ts > e.start_ts
      RETURN 1
  )
  FILTER module_count > 0
  SORT e.start_ts ASC
  LIMIT 15
  RETURN {
    epoch:   SPLIT(e.label, " — ")[1],   // strip verbose "lowRISC/ibex.git — " prefix
    date:    DATE_FORMAT(e.start_ts * 1000, "%yyyy-%mm"),
    modules: module_count
  }
```

Point out: ibex started with 14 modules in April 2015 and grew to 24+ through hundreds of named epochs. The module count fluctuates — it goes up when modules are introduced, down when they're refactored away.

Scene 3 — "Design Situations — The Déjà Vu Index" (~5 min)

Talking point: *We automatically classify every important design moment across all repos. When a new subsystem gets added, when a major refactor happens, when a release is prepared — these are indexed as "Design Situations" that can be matched against future work.*

Query 3a — Show the situation inventory

```
// What kinds of design situations exist across all repos?
FOR s IN DesignSituation
  COLLECT class = s.situation_class, repo = s.repo WITH COUNT INTO n
  SORT repo, class
  RETURN {situation_class: class, repo: repo, count: n}
```

Query 3b — Show a major refactor situation with its modules

```
// Pick a dramatic refactor situation in ibex and show what changed
FOR s IN DesignSituation
  FILTER s.repo == "lowRISC/ibex.git"
  FILTER s.situation_class == "major_refactor"
  SORT s.valid_from_ts ASC
  LIMIT 1
  LET affected_modules = (
    FOR m IN RTL_Module
      FILTER m.repo == s.repo
      FILTER m.design_epoch == s.epoch
      RETURN m.label
  )
  RETURN {
    situation:    s.label,
    epoch:        s.epoch,
    date_range:   DATE_FORMAT(s.valid_from_ts * 1000, "%yyyy-%mm-%dd"),
    tags:         s.tags,
    modules_in_epoch: LENGTH(affected_modules),
    sample_modules: affected_modules[0..4]
  }
```

Query 3c — Release preparation situations (milestone epochs)

```
// All release_prep situations — the moments before each release
FOR s IN DesignSituation
  FILTER s.situation_class == "release_prep"
  SORT s.valid_from_ts ASC
  RETURN {
    repo:    s.repo,
    release: s.epoch,
    date:    DATE_FORMAT(s.valid_from_ts * 1000, "%yyyy-%mm-%dd"),
    outcome: s.outcome,
    tags:    s.tags
  }
```

Point out: Every git release tag becomes a `release_prep` situation — automatically, without any manual annotation.

Scene 4 — "Cross-Repo Bridges — Structural DNA" (~5 min)

Talking point: *Even though OR1200 was written in 2001 and ibex was written in 2016, they share structural patterns. Here we can see which modules are architecturally equivalent across the lineage.*

Query 4a — Show all structural bridges

```
// All cross-repo structural connections
FOR e IN CROSS_REPO_SIMILAR_TO
  LET src = DOCUMENT(e._from)
  LET tgt = DOCUMENT(e._to)
  FILTER src != null AND tgt != null
  SORT e.similarity_score DESC
  RETURN {
    from_module: src.label,
    from_repo:   src.repo,
    to_module:   tgt.label,
    to_repo:     tgt.repo,
    score:       e.similarity_score,
    type:        e.similarity_type
  }
```

Point out: Modules like `cpu`, `alu`, `ctrl`, `if_stage` appear across OR1200, mor1kx, and ibex — showing the structural DNA that flows through the OpenRISC lineage into RISC-V.

Query 4b — Trace the lineage chain (EVOLVED_FROM)

```
// The architectural lineage edge — which repo evolved from which?
FOR e IN CROSS_REPO_EVOLVED_FROM
  LET src = DOCUMENT(e._from)
  LET tgt = DOCUMENT(e._to)
  RETURN {
    evolved:     src.repo,
    derived_from: tgt.repo,
    evidence:    e.lineage_type,
    score:       e.similarity_score
  }
```

Point out: marocchino `evolved_from` mor1kx — confirmed by both structural label matching AND embedding similarity of their documentation entities.

Query 4c — Cross-repo entity concepts (GraphRAG)

```
// What concepts appear in multiple repos' documentation?
FOR e IN OR1200_Golden_Entities
  LET matches = (
    FOR e2 IN MOR1KX_Golden_Entities
      FILTER LOWER(e2.entity_name) == LOWER(e.entity_name)
      RETURN e2.entity_name
  )
  FILTER LENGTH(matches) > 0
  RETURN {
    concept:    e.entity_name,
    type:       e.entity_type,
    in_or1200:  true,
    in_mor1kx:  true
  }
```

Scene 5 — "Who Knows What?" — Author Expertise Mapping (~3 min)

Talking point: *Real hardware knowledge lives in people, not just files. By linking git authors through their commits to the modules they modify, we can answer: "Who is the expert on this subsystem?" and "Which engineers span multiple repos?"*

Query 5a — Top maintainers by module ownership

```
// Who maintains the most modules across all repos?
FOR a IN Author
  LET maintained = (
    FOR m IN 1..1 OUTBOUND a MAINTAINS
      RETURN DISTINCT m.label
  )
  LET commit_repos = (
    FOR c IN 1..1 OUTBOUND a AUTHORED
      RETURN DISTINCT c.repo
  )
  FILTER LENGTH(maintained) > 0
  SORT LENGTH(maintained) DESC
  LIMIT 15
  RETURN {
    author: a.name,
    modules_maintained: LENGTH(maintained),
    repos: commit_repos,
    sample_modules: maintained[0..4]
  }
```

Point out: Stafford Horne maintains 64 modules, Pasquale Davide Schiavone maintains 63 — they are the top two experts across the entire 30-year lineage. Note how some authors span multiple repos.

Query 5b — Author expertise as a graph

> Switch to the **Graph** tab after running this to see the maintainer's footprint.

```
// Visualise a top maintainer's module ownership
LET top_author = FIRST(
  FOR a IN Author
    LET cnt = LENGTH(FOR m IN 1..1 OUTBOUND a MAINTAINS RETURN 1)
    SORT cnt DESC LIMIT 1 RETURN a
)
FOR v, e, p IN 1..2 OUTBOUND top_author MAINTAINS, BELONGS_TO_EPOCH
  LIMIT 100
  RETURN p
```

Point out: The star pattern shows one expert's footprint: Author → Module → Epoch. You can see how their work spans milestones and time periods.

Scene 6 — "The Déjà Vu Query — It's Happened Before" (~5 min)

Talking point: *Now for the showpiece: a current engineer is adding a fetch stage to their pipeline. The system instantly recognizes this situation has occurred before — in three different projects — and surfaces what happened next.*

Query 6a — Find all situations where a fetch-related module was introduced

```
// Has anyone added a fetch module before? When and in which project?
FOR s IN DesignSituation
  FILTER s.situation_class == "subsystem_addition"
  FILTER LENGTH(
    FOR t IN s.tags
      FILTER CONTAINS(LOWER(t), "fetch") OR CONTAINS(LOWER(t), "if_")
      RETURN 1
  ) > 0
  SORT s.valid_from_ts ASC
  RETURN {
    repo:    s.repo,
    epoch:   s.epoch,
    date:    DATE_FORMAT(s.valid_from_ts * 1000, "%yyyy-%mm-%dd"),
    modules: s.tags,
    outcome: s.outcome
  }
```

Query 6b — The full Déjà Vu traversal: commit → module →

epoch → situation → docs

```
// Full chain: a specific module → its epoch → situation → related doc entities
LET target_module = FIRST(
  FOR m IN RTL_Module
    FILTER m.repo == "openrisc/or1200.git" AND CONTAINS(m.label, "if_fetch")
    LIMIT 1 RETURN m
)

// If no if_fetch, use a prominent module
LET module = target_module != null ? target_module : FIRST(
  FOR m IN RTL_Module
    FILTER m.repo == "openrisc/or1200.git"
    SORT m.valid_from_ts ASC
    LIMIT 1 RETURN m
)

LET epoch = FIRST(FOR e IN DesignEpoch FILTER e.repo == module.repo AND e._key == FIRST
  FOR be IN BELONGS_TO_EPOCH FILTER be._from == module._id RETURN PARSE_IDENTIFIER(be._
) RETURN e)

LET situation = FIRST(
  FOR s IN DesignSituation
    FILTER s.repo == module.repo AND s.epoch == module.design_epoch
    LIMIT 1 RETURN s
)

LET doc_entities = (
  FOR e IN OR1200_Golden_Entities
    FILTER CONTAINS(LOWER(e.entity_name), SPLIT(module.label, "_")[-1])
    LIMIT 3
    RETURN e.entity_name
)

RETURN {
  module:      module.label,
  epoch:       module.design_epoch,
  situation:   situation.situation_class,
  situation_tags: situation.tags,
  related_docs: doc_entities,
  what_happened_next: situation.outcome
}
```

Scene 7 — "Graph Visualization" (~4 min)

Talking point: *Let me show you this in the graph viewer — you can see the temporal structure visually.*

In the ArangoDB Graph Explorer:

> **Important**
>
> The Graph Explorer's **Queries** panel only accepts AQL that returns `_id` strings of nodes already in the graph. Use those as seed nodes, then click-expand. Do NOT paste the synthetic `{vertices, edges}` query here — use the standalone Query Editor for that.

1. Navigate to **Graphs** → `IC_Temporal_Knowledge_Graph`
2. Click **Queries** in the top toolbar and paste:

```
// Seed nodes for the Graph Explorer — milestone epochs
FOR e IN DesignEpoch
  FILTER e.repo == "openrisc/mor1kx.git" AND e.epoch_type == "milestone_tag"
  SORT e.start_ts ASC
  RETURN e._id
```

3. Click each loaded node to expand it — you'll see its `RTL_Module` neighbourhood via `BELONGS_TO_EPOCH`
4. Set **depth = 2**, layout = **Force Directed** or **Hierarchical**
5. Click on an `RTL_Module` to expand further → shows `MODIFIED` edges → `GitCommit` nodes

Colour coding:

- Purple nodes = `DesignEpoch`
- Green nodes = `RTL_Module`
- Teal = `DesignSituation`
- Gold edges = `CROSS_REPO_SIMILAR_TO`

Cross-repo view in the Graph Explorer:

```
// Seed: the OR1200 Golden Entities that have cross-repo bridges
FOR e IN CROSS_REPO_SIMILAR_TO
  LIMIT 5
  RETURN e._from
```

Expand to depth 2 — you'll see two `Golden_Entity` nodes from different repos connected by a `CROSS_REPO_SIMILAR_TO` edge.

Q&A Prep — Likely Questions

Question	Answer
"How long did ETL take?"	ibex: ~2 hours for 2908 commits. OR1200 (48 commits): ~5 min. Fully idempotent — re-run skips already-ingested commits
"How are epochs determined?"	Four rules in priority order: (1) first commit, (2) git release tags, (3) 180-day time windows, (4) >15% RTL file change rate
"Can this work on proprietary repos?"	Yes — just needs git clone access. No cloud calls during ETL. GraphRAG entity extraction requires OpenAI or Ollama
"What's the ArangoDB query latency?"	Single-repo temporal queries: <200ms. Cross-repo traversals: <2s. GraphRAG community lookups: <100ms
"Can I query by author?"	Yes — 137 authors are linked to 3,796 commits via `AUTHORED` edges. 615 `MAINTAINS` edges connect authors to their primary modules (≥3 commits, ≥10% contribution). See Scene 5.
"How do RESOLVED_TO bridges work?"	We use embedding similarity to match RTL port/signal names to golden entities extracted from documentation. 214 semantic bridges currently connect code to spec.
"What about proprietary Verilog formats?"	Parser handles synthesizable SystemVerilog subset. VHDL not yet supported — planned for Phase 3 extension

Question	Answer
"How many more repos can you add?"	Schema is unbounded. Each new repo adds its prefix namespace. We've tested at 4 repos / 3,796 commits; ArangoDB OneShard handles millions
"Is the database rebuild reproducible?"	Yes — `scripts/rebuild_database.sh` runs the full pipeline end-to-end: database creation, ingestion, inference, and graph definition. Fully idempotent.

Cleanup Query (DB Sanity Check)

Run this before the demo to confirm the DB is healthy:

```
RETURN {
  GitCommit:          LENGTH(GitCommit),
  RTL_Module:         LENGTH(RTL_Module),
  DesignEpoch:        LENGTH(DesignEpoch),
  DesignSituation:    LENGTH(DesignSituation),
  Author:             LENGTH(Author),
  AUTHORED:           LENGTH(AUTHORED),
  MAINTAINS:          LENGTH(MAINTAINS),
  RESOLVED_TO:        LENGTH(RESOLVED_TO),
  CROSS_REPO_SIMILAR_TO:    LENGTH(CROSS_REPO_SIMILAR_TO),
  CROSS_REPO_EVOLVED_FROM:  LENGTH(CROSS_REPO_EVOLVED_FROM),
  OR1200_golden:      LENGTH(OR1200_Golden_Entities),
  MOR1KX_golden:      LENGTH(MOR1KX_Golden_Entities),
  IBEX_golden:        LENGTH(IBEX_Golden_Entities),
  MAROCCHINO_golden: LENGTH(MAROCCHINO_Golden_Entities)
}
```

Healthy expected values:

- GitCommit: 3,796 | RTL_Module: 6,404 | DesignEpoch: 381 | DesignSituation: 721
- Author: 137 | AUTHORED: 3,796 | MAINTAINS: 615
- RESOLVED_TO: 214 | CROSS_REPO_SIMILAR_TO: 72 | CROSS_REPO_EVOLVED_FROM: 1
- OR1200_golden: 170 | MOR1KX_golden: 39 | IBEX_golden: 546 | MAROCCHINO_golden: 120